

Trabajo Práctico Integrador: Gestión de Datos de Países en Python

Tecnicatura Universitaria en Programación a Distancia – UTN

Materia: Programación 1

29 de junio del 2026

Alumna: Luggren María de los Milagros

Índice

2. Índice
3. Introducción y Objetivos
4. Marco Teórico
5. Decisiones Técnicas y Arquitectura
6. Dificultades y Conclusiones
7. Bibliografía / Webgrafía

Introducción y Objetivos

El presente proyecto consiste en el desarrollo de una aplicación de consola en Python 3.x orientada a la gestión, filtrado, ordenamiento y análisis estadístico de un dataset de países almacenado en formato CSV. El objetivo principal radica en consolidar los fundamentos de la programación estructurada y modular, aplicando estructuras de datos dinámicas y algoritmos eficientes de ordenación en un entorno práctico y real.

Marco Teórico

Listas

Una lista en Python es una estructura de datos secuencial, mutable y ordenada que permite almacenar colecciones de elementos. En este sistema, las listas se utilizan para contener la colección completa de países cargados desde el archivo exterior, permitiendo añadir elementos mediante el método `append()` y recorrerlos mediante estructuras iterativas.

Diccionarios

Un diccionario es una estructura de datos no ordenada de parejas de clave-valor. A diferencia de las listas, el acceso a los elementos se realiza mediante llaves (keys) semánticas en lugar de índices numéricos. Cada país dentro de la aplicación es representado como un diccionario con las siguientes claves fijas: 'nombre', 'poblacion', 'superficie' y 'continente', facilitando la legibilidad del código.

Funciones y Modularización

La modularización es la técnica de diseño de software que divide un programa en partes independientes llamadas funciones. Siguiendo el principio de "una función = una responsabilidad", cada requerimiento

del sistema (como búsquedas, cálculos estadísticos o filtrados) fue aislado en funciones específicas para asegurar el mantenimiento del código y evitar la redundancia.

Estructuras Condicionales y Repetitivas

Las estructuras de control de flujo dirigen la ejecución del programa. Los condicionales (if-elif-else y match-case) permiten la toma de decisiones y el procesamiento de menús interactivas. Los bucles repetitivos (while y for) se utilizan para mantener el programa en ejecución y realizar búsquedas secuenciales dentro de las colecciones de datos.

Algoritmos de Ordenamiento (Método de Burbuja)

El ordenamiento de datos consiste en organizar los elementos de una colección siguiendo un criterio específico. En lugar de utilizar utilidades de ordenación nativas, se implementó el algoritmo de Ordenamiento de Burbuja (Bubble Sort). Este método recorre repetidamente la lista, compara elementos adyacentes y los intercambia si están en el orden incorrecto, repitiéndose hasta que la lista quede completamente organizada.

Estadísticas Básicas y Archivos CSV

El análisis estadístico incluye el cálculo de promedios aritméticos, máximos y mínimos absolutos a partir del dataset. Los archivos CSV (Comma-Separated Values) son formatos abiertos y sencillos para el almacenamiento de tablas de datos. La biblioteca estándar csv de Python nos permite leer y persistir los cambios de manera transparente utilizando codificación UTF-8.

Decisiones Técnicas y Arquitectura

El sistema se ha diseñado con un flujo lineal y robusto que prioriza la experiencia de usuario y la persistencia de datos:

Carga Inicial: Al iniciar el programa, se lee el archivo `países.csv` calculando la ruta absoluta del script para evitar fallos de entorno. Los registros se parsean y se validan las conversiones numéricas antes de guardarse en la lista en memoria.

Validación de Entradas: Se desarrollaron funciones auxiliares que impiden el ingreso de campos vacíos o valores numéricos negativos.

Persistencia Inmediata: Cualquier operación que altere la base de datos (alta o modificación) guarda automáticamente los cambios en el archivo CSV de forma inmediata para mitigar pérdidas de información.

Módulo / Función	Responsabilidad Principal
<code>cargar_datos()</code>	Lectura y validación del archivo CSV al arrancar la aplicación.
<code>algoritmo_burbuja()</code>	Lógica de ordenación genérica basada en funciones de clave.
<code>actualizar_pais()</code>	Modificación de los atributos numéricos de un registro existente.
<code>mostrar_estadisticas()</code>	Cálculo de indicadores de población, superficie y distribución geográfica.

Dificultades y Conclusiones

Durante el desarrollo surgieron desafíos técnicos específicos. El más destacado fue el manejo de errores de formato y rutas en la lectura del CSV, resuelto mediante el uso de la biblioteca `os` y validaciones con bloques `try-except`. Asimismo, la implementación manual del algoritmo de ordenación por burbuja requirió un diseño abstracto para poder ordenar tanto cadenas de texto (nombres) como valores numéricos de forma ascendente y descendente.

Como conclusión grupal, este proyecto demostró la importancia de estructurar correctamente el código desde las fases iniciales. La modularización no solo facilitó la división del trabajo, sino que garantizó que el sistema sea escalable y fácil de auditar. El uso integrado de listas, diccionarios y archivos simula de manera fidedigna los flujos de software profesionales de gestión de datos.

Bibliografía / Webgrafía

Python Software Foundation. (2026). *Python 3.x Documentation - csv module*. Recuperado de <https://docs.python.org>

Universidad Tecnológica Nacional (UTN). (2026). *Apuntes de Cátedra: Algoritmos de Ordenamiento y Estructuras de Datos - Programación 1*.